



CITS5017 Deep Learning Semester 2, 2026

Project 1

Assessed, worth 20%. Due: 11:59pm, Friday, 11th September 2026

1. Outline

In this project you will deal with a small colour image dataset containing 10 classes. Each image has $64 \times 64 \times 3$ pixels. Although the training set, validation set, and test set are all perfectly balanced, there are only 370 training instances available for each class. Your tasks for this project are:

- train a CNN model including hyperparameter tuning;
- retrain a pretrained **NASNetMobile** model; and
- compare the performances of these two models.

This project is a good practical exercise to test your understanding of the techniques covered in Chapters 10, 11, and 14.

2. Submission

Name your Jupyter Notebook file as **Surname_FirstName-proj1.ipynb** (please replace **Surname** and **FirstName**¹ by your own surname and first name). Please save your two trained models to the files **Surname_FirstName-CNN.keras** and **Surname_FirstName-NASNetMobile.keras**. You need to make sure that your models were saved in the same directory (i.e., not under a subdirectory or somewhere else) with your Notebook file, as this is what we will assume where they are in our marking process.

To create the zip file for submission, use the standard **zip** program². Your zip file can be created using one of the following methods:

- Type the following command in a terminal window:
zip Surname_FirstName.zip Surname_FirstName-proj1.ipynb *.keras.
- In your File Exploration window, navigate to the directory where your ipynb and the two keras files are, highlight them, right click, and select “Add to zip” in the drop-down menu; enter the zip file name in the pop-up window.

Notes:

- Your zip file should **not** contain any subdirectories. It should contain **only** the three files that we need for marking your project.
- As your zip file is quite large (due to the size of the two keras files), you should ensure **you have stable internet connection and reserve sufficient time for uploading your zip file.**

When your zip file is successfully submitted to LMS, you should see an acknowledgement screen. You can take a screen shot and keep the image as evidence of your submission. Do **not** send your zip file via

¹You can include your middle name if you like. It does not matter. We just want to be able to distinguish all the students' submitted files when we put the whole class's submissions together in one large directory when we mark all the projects. **NOTE:** If your surname and/or first name contain special characters, such as apostrophes, '/', etc, you will have to omit them or replace them by underscores or hyphens as these characters have special meanings: apostrophes are considered the same as single quotes; '/' is used to denote directories.

²To save us time, **do not use other zip programs such as gzip, tar, WinZip, 7-zip, etc**

email to the Unit Coordinator. Do **not** upload your submission onto Google Drive or OneDrive and then send the link to the Unit Coordinator. **Only files submitted to LMS will be marked.**

3. The Image Dataset

Three pickle files, *train-2026.pkl*, *valid-2026.pkl*, and *test-2026.pkl*, are supplied on the LMS page of the unit. Download these three files and save them in the **same directory** where you would develop your code. Use the following code to load the data:

```
def load_data(pickle_file):
    import pickle
    with open(pickle_file, "rb") as f:
        dict = pickle.load(f)
    return dict[b"data"], dict[b"labels"], dict[b"class_names"]

# now call load_data to build the training, validation, and test sets
X_train, y_train, class_names = load_data("./train-2026.pkl")
X_valid, y_valid, _ = load_data("./valid-2026.pkl")
X_test, y_test, _ = load_data("./test-2026.pkl")
```

You should find that the shapes of `X_train`, `X_valid`, and `X_test` are (3700, 64, 64, 3), (800, 64, 64, 3), and (500, 64, 64, 3), respectively, i.e., for each class, there are 370 training instances, 80 validation instances, and 50 test instances. All of them are of type `uint8` (8-bit unsigned integer, just enough to represent any pixel value in the range $0 \dots 255$). All the instances are stored in increasing class IDs from 0 to 9. You should randomly shuffle³ the training set (and optionally the validation set) to make sure that the minibatches used in the training process later on have a good mix of instances from the 10 classes.

4. Tasks

- (i) Write a small function called `display_images` that would take in appropriate arguments so that it can be used to display 30 randomly sampled images from the arguments. This function should be called 3 times – for the training, validation, and test sets. The figures displayed by the function should show the corresponding class name of each image.

Note: you should design this function with additional optional arguments that would allow you to reuse it for the **Comparisons** section of your CNN model and your retrained NASNetMobile model later on.

- (ii) **Structure of your code**

In the marking process, we want to bypass the hyperparameter tuning process of your CNN model. So, after displaying the images using your `display_images` function, follow the structure given below in your Jupyter Notebook file:

```
hyperparameter_tuning = True    # this is a boolean variable

if hyperparameter_tuning is True then
    perform the hyperparameter tuning for your CNN model (part (iii))
otherwise
    load your CNN model from Surname_FirstName-CNN.keras
    show the CNN model's architecture

design and train your NASNetMobile model (part (iv))
compare the performance of the two models (part (v))
```

³for instance, using `numpy.random.permutation` or `numpy.random.shuffle`.

After you have obtained the best CNN model from hyperparameter tuning, you should manually change `hyperparameter_tuning` to `False` so that subsequent runs of your Jupyter Notebook file will bypass hyperparameter tuning. You should also note down the total tuning time returned by the Keras tuner and report it in a markdown cell for the **Comparisons** section later on.

(iii) Implementation and hyperparameter tuning of a CNN model

- Design a CNN model that has 4 convolutional layers with an appropriate activation function for each layer. Your CNN model should have 4 pooling layers and a *drop out* layer. You could optionally include other suitable layers. Near the top of the network, rather than using a *flattening* layer, you are suggested to use a *global average pooling* layer as it has been shown to not only work better but also help reduce the number of trainable parameters. After that, your model should have a *dense* layer before the output layer.

Choose a suitable optimizer and a suitable learning rate scheduler for your CNN model.

- You should use the `BayesianOptimization` tuner from `keras_tuner`⁴ to help you perform hyperparameter tuning for the following **six** hyperparameters:
 - the number of filters for each of the 4 convolutional layers;
 - the drop-out rate for the *drop out* layer;
 - the number of neurons for the *dense* layer before the output layer.

For each of the hyperparameters above, you will need to choose a suitable range of values⁵ for the tuner. As the training set size is quite small, you should set the ranges of these hyperparameters carefully so that the total number of trainable parameters for the whole network is below 30,000.

You should set the `max_trials` parameter for the tuner to 15. In each trial, you should get the tuner train the network for 120 epochs using the validation set for early stopping.

- You may need to run the tuner several times (each time involves 15 trials), compare the performance of the best model found each time. For the complexity of this dataset and the small training set size, if your CNN model can achieve a test accuracy above 50%, it is sufficient for the project.

You will need to include the hyperparameter tuning code in your Jupyter Notebook file. Tidy up your code cells and keep only the best tuning output.

- Save the best model that you have found to the file **Surname FirstName-CNN.keras**.
- Use an appropriate function from `tensorflow.keras` to show a summary of your CNN architecture.

(iv) Fine tuning the NASNetMobile model

- Use the pretrained model *NASNetMobile* from the `tf.keras.applications` package as your base model⁶. In terms of model complexity, NASNetMobile is reasonably small among all the available pretrained models in the package. Like other models, it has been trained using the ImageNet dataset which has 1000 classes. Replace the output layer so that the network would be suitable for your 10-class classification problem. Before the output layer, your model should have a *global average pooling 2D* layer followed by a *dense* layer. Ensure that your dense layer has 80 or fewer neurons so that your model is not exceedingly large. Use the learning curves plot (see later) to help you judge whether you have the overfitting issue.

⁴see Chapter 10 of the textbook and the **Fine-tuning Neural Network Hyperparameters** section in `10_neural_nets_with_keras.ipynb` from Géron.

⁵Obviously, you would want more filters as you move deeper into the network.

⁶see the **Using Pretrained Models for Keras** section in `14_deep_computer_vision_with_cnns.ipynb`.

- Display the summary for the first four layers of your network. Repeat this step for the last four layers.
- Freeze all the layers from the base network and train only your added layers. Use a suitable learning rate scheduler (which can be the same as the one you used for the CNN model). Train your model for 120 epochs with early stopping.
- Turn off verbose output from the training process; instead, display the learning curves (training loss, validation loss, training accuracy, and validation accuracy versus epochs).
- Note down the time⁷ taken to train the network in a Python variable for the **Comparisons** section later on.
- Save your trained model to **Surname_FirstName-NASNetMobile.keras**.

(v) **Comparisons**

- Reload both models from the saved keras files.
- Report the time taken to tune the hyperparameters for your CNN model and the time taken to train your NASNetMobile model in a markdown cell.
- Report the complexity (i.e., number of trainable parameters) of both models in a code cell. Note that, for your adapted NASNetMobile model, the frozen layers of the base model should not be included in the counting.
- Use code cells to report the performances of the two models in terms of their precision, recall, and F1 score for each of the 10 classes for the test set. Also should be reported are their overall test accuracies.
- Write Python code to show the confusion matrices of the two models for the test set (the Scikit-learn library has some useful functions that you can use).
- Finally, compare the two models by showing up to 20 example images from the test set where (i) both models correctly classified, (ii) both models incorrectly classified; (iii) CNN classified correctly but the adapted NASNetMobile model did not; and (iv) adapted NASNetMobile model classified correctly but CNN did not.
- Apart from providing the code cells for all the subtasks above, provide also a summary markdown cell about the two models and their performances.

(vi) **Markdown cells**

Throughout your entire Jupyter Notebook file, you should provide appropriate markdown cells alongside the code cells to summarise/explain your implementation, assumptions that you made, and observations from the outputs. You should divide your Jupyter Notebook file into suitable sections and subsections with meaningful headings. Brief comments should also be included in the code cells; however, you should avoid over-commenting your code⁸.

5. Google Colab users

This section is relevant for Google Colab users only. If you use Google Colab for the project, you will need to upload the three pickle files to your Google Drive and have a couple of extra code cells to connect your Google Colab session with your Google Drive account. In your final submitted file, you will need to remove those code cells as we cannot access your Google Drive account when marking your project. Penalty will be applied if we need to modify your code in order to run it.

⁷You can use any of the Python modules, such as `time` or `timeit`, to help you record the training time.

⁸It is recommended to have just a couple of lines to comment a block (several lines) of code or comment only where needed.

6. Kaya users

As Kaya is still being set up by the UWA HPC team. Further information will be provided here when available.

7. Penalty on late submissions

See the URL below about late submission of assignments:

https://ipoint.uwa.edu.au/app/answers/detail/a_id/2711/~consequences-for-late-assignment-submission

Late submissions will be calculated at 5% of the total mark per day for the first 7 days (including weekends and public holidays) after which the project is not accepted. If your submission is made within two days (48 hours), no penalty will be applied. However, from the third day onward, penalty will start from 10%.

8. Plagiarism

Do not blindly copy examples that you found on the internet. Many of these examples are often too complex and they do not meet the project specification here. **Significant penalty** will be imposed on your mark if your code is found to be copied from the internet. You should be able to complete this project by following examples in the lectures, textbook, and sample codes from the author Géron.

You should attempt the project by yourself. Collusion with (an)other student(s) is considered to be serious academic misconduct and can cause you to be suspended or expelled from the unit. Please see <https://www.uwa.edu.au/students/my-course/student-conduct> for more details.